

WEBASSEMBLY PROJECT

透過 WebAssembly 運用 C 語言 在瀏覽器內實現高效能影像處理

高效像素運算與 AI Agent 協作開發實踐

專案簡介與核心挑戰

效能開發痛點

傳統 JavaScript 在 Canvas 操作上，若涉及複雜的逐 Byte 圖片處理，其動態類型的特性與垃圾回收機制往往難以應付極致的運算需求。

WebAssembly 解決方案

將繁重的運算核心交給由 C 編譯而成的 WebAssembly (Wasm)。透過靜態類型與二進位執行，其效能表現能遠超傳統 JS 操作。

 重點：將運算負擔從 JavaScript 轉移至近原生的 Wasm 核心

核心型別：Uint8 的無縫對接



C 側：uint8_t

- **Unsigned 8-bit Integer**
- 佔用 1 Byte, 範圍 0 ~ 255
- 定義在 `<stdint.h>`
- 精準表示 RGBA 的單一數值



JS 側：Uint8ClampedArray

- 來自 `ImageData.data`
- 同樣為 1 Byte 的數值陣列
- 不需複雜轉譯即可讀寫
- 提供高效的 `TypedArray` 視圖

C 側技術：結構體與指針強轉型

```
typedef struct {
    uint8_t r;
    uint8_t g;
    uint8_t b;
    uint8_t a;
} Pixel;

void applyGrayscale(uint8_t* ptr) {
    // 將 1-byte 指針強轉為 4-byte 結構
    Pixel* img = (Pixel*)ptr;
    ...
}
```

- **無間隙對齊**：Pixel 大小恰好為 4 Bytes，與 Canvas RGBA 排列一致。
- **記憶體連續性**：利用二進位排列特性，不需 Padding 即可直接映射。
- **高可讀性**：轉型後可用 `img[i].r` 直接操作，兼顧效能與開發體驗。

數據橋樑: Linear Memory 流程

1. 分配 (Alloc)

JS 調用 `_allocMemory`。回傳的指針是 Heap 中的 Index。

2. 寫入 (Set)

JS 執行 `HEAPU8.set(pixels, ptr)` 將數據存入 Wasm Heap。

3. 取出 (Subarray)

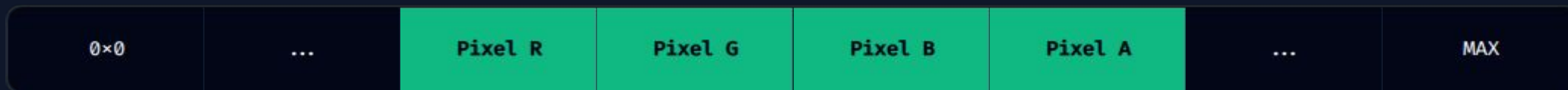
處理後使用 `subarray()` 獲取指向結果的視圖。

4. 釋放 (Free)

手動執行 `_freeMemory(ptr)`，避免記憶體洩漏。

數據橋樑圖解：指針與 HEAPU8

WebAssembly.Memory (HEAPU8)



↑ ptr (Return value of `_alloc`)

JS 調用 C 函數時，傳遞的是這個 **Index 整數**。Wasm 內部則將此整數視為**記憶體位址**。

關鍵實踐：手動記憶體管理

- **手動釋放的重要性**：Wasm 記憶體不受 JS GC 控制，必須手動回收。
- **Try-Finally 模式**：確保在每次批次處理後，無論成功失敗皆釋放指針。
- **預防分頁崩潰**：在大規模批次處理（數百張圖片）時，未釋放的記憶體將迅速導致 OOM。



手動管理 = 安全效能

AI Agent 協作開發：自動化閉環

自動化除錯流程

利用 Agent 自主架設伺服器並控制 Chrome 進行測試，能即時檢測 Console 報錯（如 Wasm 定義錯誤或記憶體溢出）。

高效修復循環

Agent 具備讀取日誌與自我修正代碼的能力。人類不再需要擔任「人肉測試機」，而是專注於系統架構決策。

AI 協作新範式：規格即生 產力



精準的架構界線

C 側專注無狀態運算，JS 側處理狀態與 I/O。
清晰的職責劃分是 AI 生成高品質代碼的前提。



明確的需求規格

定義清楚的二進位對齊與內存交換規範，讓
Agent 能在「正確的框架」內自主探索最優
解。



感謝您的聆聽

 Project Link:

<https://github.com/chuan13/webassembly-c-demo>

 線上試用 Demo:

<https://chuan13.github.io/webassembly-c-demo/>